Shor's Algorithm

Factoring by Quantum Period Finding

A step-by-step chapter: from modular arithmetic to a CUDA-Q implementation

What you will be able to do after this chapter

- compute gcds, inverses, totients and multiplicative orders by hand, and use repeated squaring and the Chinese remainder theorem;
- explain why factoring is hard classically (trial division, Pollard rho, GNFS) and why RSA depends on it;
- prove the reduction of factoring to order finding and decide when an attempt succeeds or must be repeated;
- derive the order-finding circuit: registers, controlled modular multiplication, phase kickback, inverse QFT and phase estimation;
- turn a measured value into the period with continued fractions and verify the factors;
- read and modify `Shor.ipynb`, estimate qubits, gates and memory, and interpret recorded, re-executed and noisy results honestly;
- discuss resource estimates, fault tolerance, RSA, elliptic curves and post-quantum cryptography;
- solve every type of question in the accompanying 80-question Shor assessment.

Companion material in the same repository: `presentations/Shor_Lecture.pdf` (slides), `Shor.ipynb` (CUDA-Q notebook) and `assessments/` (question paper and answer book).

Contents

Shor's Algorithm — Factoring by Quantum Period Finding **4**

1 Number theory toolkit . . . 4

1.1 Divisibility, gcd and the Euclidean algorithm . . . 4

1.2 Modular arithmetic, inverses and permutations . . . 5

1.3 Euler's totient, orders and the Carmichael function . . . 5

1.4 Modular exponentiation by repeated squaring . . . 5

1.5 Chinese remainder theorem and square roots of one . . . 6

2 Classical factoring and RSA . . . 6

2.1 How hard is factoring? . . . 6

2.2 RSA in five lines . . . 6

3 From factoring to order finding . . . 7

4 The quantum toolkit for Shor . . . 7

4.1 Registers and the reversible exponentiation . . . 7

4.2 Eigenvectors of U_a and phase kickback . . . 8

4.3 Gates you meet in the circuit . . . 8

5 Period finding and the quantum Fourier transform . . . 8

5.1 The circuit . . . 8

5.2 What the inverse QFT does to a comb . . . 9

5.3 The QFT circuit . . . 9

6 Quantum phase estimation . . . 9

7 Continued fractions and classical post-processing . . . 10

7.1 Why the measurement is not the period . . . 10

7.2 Continued fractions . . . 10

8 Fully worked problems . . . 11

9 Circuit architecture and resources . . . 12

9.1 Register sizes and qubit counts . . . 12

9.2 Gates, depth and connectivity . . . 12

10 The implementation in `Shor.ipynb` . . . 12

10.1 Structure and code . . . 12

10.2 Results . . . 13

11 Noise, approximations and limitations 13
11.1 Noisy and approximate circuits (illustrative experiments) 13
11.2 Limitations 14
12 Cryptography and outlook 14
13 Problem-solving strategies and question map 14
13.1 Strategy by question format 14
13.2 Fast checks that catch most mistakes 15
13.3 Map: assessment questions → sections of this chapter 15
14 Practice problems 15

Shor's Algorithm — Factoring by Quantum Period Finding

How to use this chapter

The chapter is built as a staircase. Sections 1–3 are the *number theory and classical background* (modular arithmetic, orders, classical factoring, RSA, and the reduction of factoring to order finding). Sections 4–7 are the *quantum part* (registers and unitary multiplication, period finding with the QFT, phase estimation, continued fractions). Section 8 solves complete problems. Sections 9–12 cover *circuits and resources*, the *implementation* in `Shor.ipynb`, *noise and limitations*, and *cryptography*. Section 13 collects problem-solving strategies and maps every assessment question to the section that teaches it; Section 14 has 16 practice problems with answers.

Each section has: the idea in words, the precise statement, a derivation, **worked examples** solved step by step, **common traps** and a short checklist. Statements are marked as *general theory*, *the notebook's implementation*, or *illustrative experiments*; "previously recorded" results are the outputs stored in `Shor.ipynb`, "re-executed" results were produced for this package with `presentations/shor_source/make_figures.py`. All simulations ran on a CPU with CUDA-Q 0.16.0; **no physical quantum processor and no GPU were used**. Every number in the assessment was recomputed by `assessments/shor_test_source/verify_answers.py`.

Definition: Conventions used everywhere in this chapter

- $a \bmod N \in \{0, \ldots, N-1\}$; $a \equiv b \pmod N$ means $N \mid a - b$. $n = \lceil \log_2(N+1) \rceil =$ `N.bit_length()`.
- The **order** $r = \mathrm{ord}_N(a)$ is the least $r > 0$ with $a^r \equiv 1 \pmod N$ (defined when $\gcd(a, N) = 1$).
- Circuit: t counting qubits (notebook: $t = 2n$), n work qubits in $|1\rangle$, controlled $U_c\,|y\rangle = |cy \bmod N\rangle$ ($y < N$; identity for $y \geq N$).
- The measured integer $y \in \{0, \ldots, 2^t - 1\}$ is read with the **most significant bit first**. $\mathrm{QFT}\,|x\rangle = 2^{-t/2} \sum_y e^{2\pi i x y / 2^t} |y\rangle$.
- $R_1(\lambda) = \mathrm{diag}(1, e^{i\lambda})$; `cr1` is its controlled version. Memory: 16 bytes per complex amplitude.

1 Number theory toolkit

1.1 Divisibility, gcd and the Euclidean algorithm

Every integer $N > 1$ is a product of primes in exactly one way. The **greatest common divisor** $\gcd(a, b)$ is the largest integer dividing both; a, b are **coprime** if it is 1. The Euclidean algorithm uses $\gcd(a, b) = \gcd(b, a \bmod b)$ and needs $O(\log \min(a, b))$ divisions, so it is fast even for 2048-bit

numbers. Running it backwards (*extended Euclid*) writes $\gcd(a, b) = ua + vb$.

Worked example 1.1: $\gcd(1071, 462)$ and an inverse

Step 1. $1071 = 2 \cdot 462 + 147$; $462 = 3 \cdot 147 + 21$; $147 = 7 \cdot 21 + 0$. The last non-zero remainder is $\gcd = 21$.

Step 2. Inverse of 3 modulo 20: $20 = 6 \cdot 3 + 2$, $3 = 1 \cdot 2 + 1$, so $1 = 3 - 2 = 3 - (20 - 6 \cdot 3) = 7 \cdot 3 - 20$: $3^{-1} \equiv 7$. Check: $21 \equiv 1$.

1.2 Modular arithmetic, inverses and permutations

Sums and products may be reduced modulo N at every step. a has an inverse modulo N iff $\gcd(a, N) = 1$. Then $y \mapsto ay \bmod N$ is a **bijection** of $\{0, \ldots, N-1\}$ (it has the inverse $y \mapsto a^{-1}y$). If $\gcd(a, N) = d > 1$ two inputs collide, e.g. $3 \cdot 0 \equiv 3 \cdot 5 \pmod{15}$.

Key result: Why coprimality matters for the quantum circuit

$U_a |y\rangle = |ay \bmod N\rangle$ is a permutation matrix, hence unitary, exactly when $\gcd(a, N) = 1$. Shor's circuit can therefore only be built for coprime bases — and a non-coprime base already reveals the factor $\gcd(a, N)$.

1.3 Euler's totient, orders and the Carmichael function

$\varphi(N)$ counts the residues coprime to N: $\varphi(p) = p - 1$, $\varphi(pq) = (p-1)(q-1)$ for distinct primes, $\varphi(p^k) = p^{k-1}(p-1)$. **Euler's theorem**: $a^{\varphi(N)} \equiv 1$ for $\gcd(a, N) = 1$ (Fermat's little theorem for prime N). The order r divides every exponent e with $a^e \equiv 1$; in particular $r \mid \lambda(N) \mid \varphi(N)$, where the **Carmichael function** is $\lambda(pq) = \operatorname{lcm}(p-1, q-1)$ for odd primes. Hence $f(x) = a^x \bmod N$ is periodic with period exactly r.

Worked example 1.2: Orders modulo 15 and 35

Step 1. $\varphi(15) = 8$, $\lambda(15) = \operatorname{lcm}(2, 4) = 4$. Powers of 2: $2, 4, 8, 1$ ($r = 4$); of 4: $4, 1$ ($r = 2$); of $14 \equiv -1$: $r = 2$.

Step 2. $\lambda(35) = \operatorname{lcm}(4, 6) = 12$, so the possible orders are $1, 2, 3, 4, 6, 12$; an order such as 5 is impossible because $5 \nmid 12$.

Step 3. 3 modulo 35: $\operatorname{ord}_5(3) = 4$, $\operatorname{ord}_7(3) = 6$, so $\operatorname{ord}_{35}(3) = \operatorname{lcm}(4, 6) = 12$ (CRT, below).

1.4 Modular exponentiation by repeated squaring

Write $x = \sum_j x_j 2^j$. Then $a^x = \prod_j (a^{2^j})^{x_j}$: square $n - 1$ times and multiply in the powers with $x_j = 1$. For an m-bit exponent with w one-bits this costs $(m-1) + (w-1)$ modular multiplications, e.g. 14 for $x = 1000 = 1111101000_2$, instead of 999.

Worked example 1.3: $3^{13} \bmod 17$

$13 = 1101_2$. $3^2 = 9$, $3^4 = 81 \equiv 13$, $3^8 \equiv 13^2 = 169 \equiv 16$. $3^{13} = 3^8 \cdot 3^4 \cdot 3 \equiv 16 \cdot 13 \cdot 3 = 624 = 36 \cdot 17 + 12 \equiv 12$.

The quantum circuit uses the same structure: counting qubit j controls a multiplication by the classically precomputed $a^{2^j} \bmod N$.

1.5 Chinese remainder theorem and square roots of one

For coprime p, q, $x \bmod pq$ corresponds one-to-one to the pair $(x \bmod p, x \bmod q)$. Modulo an odd prime, $x^2 \equiv 1$ has only $x \equiv \pm 1$; modulo pq the four combinations $(\pm 1, \pm 1)$ give **four** square roots of 1. For 15: $1, 4, 11, 14$.

Key result: Non-trivial square roots factor N

If $x^2 \equiv 1$ and $x \not\equiv \pm 1 \pmod N$, then $N \mid (x-1)(x+1)$ but N divides neither factor, so $\gcd(x-1, N)$ and $\gcd(x+1, N)$ are non-trivial factors. Example: $x = 4$, $N = 15$: $\gcd(3, 15) = 3$, $\gcd(5, 15) = 5$.

Common trap

$\varphi(N)$ is an exponent that works, but it is rarely the order: $\mathrm{ord}_{15}(4) = 2 \neq 8$. And the order is the *smallest* exponent — multiples of r such as $2r$ also satisfy $a^{2r} \equiv 1$.

Checklist

Can you: run Euclid; find an inverse; compute φ and λ; list powers to find an order; use $r \mid \lambda(N)$ to exclude orders; find all square roots of 1 modulo pq?

2 Classical factoring and RSA

2.1 How hard is factoring?

method	**cost for an n-bit N**	**type**
trial division up to $\sqrt{N}$	$O(2^{n/2})$	exponential
Pollard's rho	$O(N^{1/4}) = O(2^{n/4})$ expected	exponential
elliptic-curve method (ECM)	sub-exponential in the size of the smallest factor	good for small factors
quadratic sieve	$\exp\left((1+o(1))\sqrt{\ln N \ln\ln N}\right)$	sub-exponential
general number field sieve	$\exp\left((64/9)^{1/3}(\ln N)^{1/3}(\ln\ln N)^{2/3}(1+o(1))\right)$	sub-exponential
Shor (quantum, fault-tolerant)	$O(n^3)$ gates (schoolbook arithmetic)	polynomial

The record for a hard RSA-type number factored classically is RSA-250 (829 bits, 2020) with the GNFS, about 2700 core-years. Trial division would need $\approx 2^{1024}$ divisions for RSA-2048.

Common trap

"All classical factoring algorithms are exponential" is false: the quadratic sieve and the GNFS are sub-exponential. Shor's advantage is *super-polynomial* over the best known classical algorithm, not over trial division only. Factoring is also not known to be NP-complete.

2.2 RSA in five lines

Choose primes p, q, publish $N = pq$ and e with $\gcd(e, \varphi(N)) = 1$; keep $d = e^{-1} \bmod \varphi(N)$. Encrypt $c = m^e \bmod N$, decrypt $m = c^d \bmod N$. Factoring N gives $\varphi(N)$, hence d.

Worked example 2.4: Toy RSA

$p = 5$, $q = 11$, $N = 55$, $\varphi = 40$, $e = 7$. $d = 23$ since $7 \cdot 23 = 161 = 4 \cdot 40 + 1$. For $m = 2$: $c = 2^7 = 128 \equiv 18$, and $18^{23} \equiv 2 \pmod{55}$.

3 From factoring to order finding

Method: Shor's reduction (classical)

Input: odd composite N that is not a prime power.

1. Choose a uniformly in $\{2, \ldots, N-2\}$. If $d = \gcd(a, N) > 1$, output d (lucky, no quantum step).
2. Find $r = \mathrm{ord}_N(a)$ *(the only quantum step).*
3. If r is odd or $a^{r/2} \equiv -1 \pmod{N}$: the attempt **fails** — go to 1.
4. Otherwise output $\gcd(a^{r/2} - 1, N)$ and $\gcd(a^{r/2} + 1, N)$.

Why it works. $x = a^{r/2}$ satisfies $x^2 = a^r \equiv 1$. $x \not\equiv 1$ because r is minimal; if also $x \not\equiv -1$, x is a non-trivial square root of 1 (Section 1.5). **How often it works.** If N has $k \geq 2$ distinct odd prime factors, a random coprime a succeeds with probability at least $1 - 2^{-(k-1)}$; for an RSA modulus at least $\frac{1}{2}$, so the expected number of attempts is at most 2.

Worked example 3.5: Successful and failing bases for $N = 21$

Step 1. $a = 2$: $2, 4, 8, 16, 11, 1 \Rightarrow r = 6$; $2^3 = 8 \not\equiv 20$; $\gcd(7, 21) = 7$, $\gcd(9, 21) = 3$. **Success**: $21 = 3 \times 7$.

Step 2. $a = 4$: $4, 16, 1 \Rightarrow r = 3$, odd. **Failure** — choose another a.

Step 3. $a = 5$: $5, 4, 20, 16, 17, 1 \Rightarrow r = 6$, but $5^3 \equiv 20 \equiv -1$: gcds 1 and 21. **Failure**.

Step 4. $a = 6$: $\gcd(6, 21) = 3$. **Lucky** factor without quantum computation.

Common trap

The order is *not* a factor: for $N = 15$, $a = 7$, $r = 4$ does not divide 15. And a correct order does not guarantee factors ($a = 14$ modulo 15: $r = 2$, $14 \equiv -1$). Report failed attempts as failures.

Excluded cases. Even N: factor 2. Prime N: nothing to do (Miller–Rabin). Prime powers $N = p^k$: $\mathbb{Z}_{p^k}^*$ is cyclic, so the only square roots of 1 are ± 1 and the reduction always fails; integer k-th roots find p classically. `Shor.ipynb` handles all three classically.

4 The quantum toolkit for Shor

4.1 Registers and the reversible exponentiation

The **counting register** (t qubits) holds the exponent x; the **work register** (n qubits) holds values modulo N. Because $x \mapsto a^x \bmod N$ is not injective ($a^{x+r} = a^x$), it cannot act in place; instead

$$|x\rangle\,|1\rangle \longmapsto |x\rangle\,|a^x \bmod N\rangle$$

is realised by multiplying the work register by $a^{x_j 2^j}$ for each bit — a product of controlled $U_{a^{2^j}}$. After Hadamards on the counting register, the result is the entangled state $2^{-t/2}\sum_x |x\rangle\,|a^x \bmod N\rangle$.

Common trap

"Quantum parallelism evaluates f on all x, so we can read off the period" — a measurement returns one random pair $(x, f(x))$, no more than a classical evaluation. The period must be extracted by **interference**.

4.2 Eigenvectors of U_a and phase kickback

On the orbit $|1\rangle \to |a\rangle \to \cdots \to |a^{r-1}\rangle \to |1\rangle$, U_a is a cyclic shift of length r. Its eigenvectors and eigenvalues are

$$|u_s\rangle = \frac{1}{\sqrt{r}}\sum_{k=0}^{r-1} e^{-2\pi i s k/r}\,|a^k \bmod N\rangle\,, \qquad U_a\,|u_s\rangle = e^{2\pi i s/r}\,|u_s\rangle\,, \qquad s = 0,\dots,r-1,$$

and $\frac{1}{\sqrt{r}}\sum_s |u_s\rangle = |1\rangle$ (the sum of r-th roots of unity vanishes unless $k = 0$). A controlled-U acting on an eigenvector leaves the target unchanged and **kicks back** the phase onto the control: $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\,|u\rangle \mapsto \frac{1}{\sqrt{2}}(|0\rangle + e^{i\phi}\,|1\rangle)\,|u\rangle$. With U^{2^j} the kicked phase is $2^j\phi$.

Worked example 4.6: Eigenvalues of U_2 modulo 15 and a kicked-back phase

Step 1. Orbit $1 \to 2 \to 4 \to 8 \to 1$: $r = 4$, eigenvalues $e^{2\pi i s/4} = 1, i, -1, -i$.

Step 2. Control in $|+\rangle$, phase $\phi = 2\pi/3$: the control becomes $\frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i/3}\,|1\rangle)$ with $\langle X\rangle = \cos\phi = -\frac{1}{2}$.

4.3 Gates you meet in the circuit

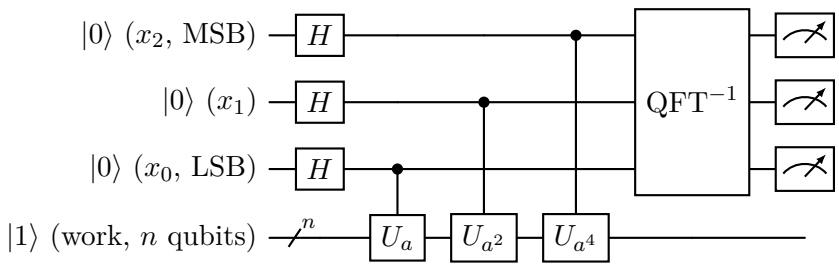
H (superposition), X (prepares $|1\rangle$), controlled-U_c (multiplication), SWAP, and $R_1(\lambda)$ with its controlled version: $R_1(\pi) = Z$, $R_1(\pi/2) = S$, $R_1(\pi/4) = T$, $R_1(-\pi/2) = S^\dagger$. $H^{\otimes n}\,|x\rangle = 2^{-n/2}\sum_y (-1)^{x\cdot y}\,|y\rangle$. A controlled SWAP (Fredkin) is CNOT · Toffoli · CNOT = 8 CNOTs with the 6-CNOT Toffoli.

Checklist

Explain why U_a is unitary; write $|u_s\rangle$; show $\sum_s |u_s\rangle \propto |1\rangle$; compute a kicked-back phase for U^{2^j}; name $R_1(\pi), R_1(\pi/2), R_1(\pi/4)$.

5 Period finding and the quantum Fourier transform

5.1 The circuit



($t = 3$ shown.) The states are

$$|\psi_1\rangle = 2^{-t/2}\sum_x |x\rangle\,|1\rangle\,, \qquad |\psi_2\rangle = 2^{-t/2}\sum_x |x\rangle\,|a^x \bmod N\rangle = 2^{-t/2}\sum_v \Big(\sum_{x:\,a^x\equiv v} |x\rangle\Big)\,|v\rangle\,.$$

For each value v the exponents form an arithmetic progression $x_0, x_0 + r, x_0 + 2r, \ldots$ (a *comb*). Measuring the work register (or not — the statistics of y are the same) leaves one comb in the counting register.

5.2 What the inverse QFT does to a comb

The inverse QFT maps $|x\rangle \mapsto 2^{-t/2}\sum_y e^{-2\pi i xy/2^t}\,|y\rangle$. For a comb with M teeth,

$$P(y) = \frac{1}{2^t M}\Big|\sum_{m=0}^{M-1} e^{-2\pi i m r y/2^t}\Big|^2 = \frac{\sin^2(\pi M r y/2^t)}{2^t M \sin^2(\pi r y/2^t)},$$

independent of the offset x_0, and large only for $y \approx k2^t/r$. If $r \mid 2^t$ the peaks are exact: $P(k2^t/r) = 1/r$ for $k = 0, \ldots, r-1$ and 0 elsewhere. Otherwise each peak has finite width and the integer nearest to $k2^t/r$ is obtained with probability at least $\frac{4}{\pi^2}\cdot\frac{1}{r}$.

Worked example 5.7: $N = 15$ and $N = 21$ with the notebook's register sizes

Step 1. $N = 15$, $a = 7$ (or 2), $t = 8$: $r = 4$ divides 256; outcomes $0, 64, 128, 192$ with probability exactly $\frac{1}{4}$ each.

Step 2. $N = 15$, $a = 11$: $r = 2$; outcomes $0, 128$, each $\frac{1}{2}$.

Step 3. $N = 21$, $a = 2$, $t = 10$: $1024/6 = 170.67$; peaks near $0, 171, 341, 512, 683, 853$. $P(0) = P(512) = \frac{1}{6}$ exactly; $P(171) = 0.1140$ and $P(170) = 0.0285$ (re-executed state vector).

5.3 The QFT circuit

t Hadamards, $t(t-1)/2$ controlled phases $R_1(\pm 2\pi/2^k)$ and $\lfloor t/2 \rfloor$ SWAPs: $O(t^2)$ gates. The notebook's inverse QFT performs the SWAPs first and then, for each qubit j from the last to the first, the controlled phases `cr1(-pi/2**(k-j))` followed by H; for $t = 8$ that is 4 SWAPs, 28 controlled phases and 8 Hadamards.

Common trap

The QFT is *not* Shor's algorithm. It neither computes $a^x \bmod N$ (the expensive part) nor outputs r or the factors. Its role is to turn periodicity into peaks that a single measurement can sample.

6 Quantum phase estimation

Phase estimation (QPE) with t counting qubits estimates φ in $U\,|u\rangle = e^{2\pi i\varphi}\,|u\rangle$. After the controlled U^{2^j} gates the counting register is $2^{-t/2}\sum_x e^{2\pi i x\varphi}\,|x\rangle$, and after the inverse QFT

$$P(y) = \Big|\frac{1}{2^t}\sum_{x=0}^{2^t-1} e^{2\pi i x(\varphi - y/2^t)}\Big|^2 = \frac{\sin^2(\pi 2^t \delta)}{4^t \sin^2(\pi\delta)}, \qquad \delta = \varphi - \frac{y}{2^t}.$$

If $2^t\varphi$ is an integer, $y = 2^t\varphi$ with probability 1; otherwise the nearest y has probability $\geq 4/\pi^2 \approx 0.405$.

Key result: Order finding is phase estimation of U_a

With the work register in $|1\rangle = \frac{1}{\sqrt{r}}\sum_s |u_s\rangle$, QPE returns $y \approx 2^t s/r$ for a uniformly random $s \in \{0, \dots, r-1\}$: the same peaks as in the period-finding picture. The controlled $U_a^{2^j} = U_{a^{2^j} \bmod N}$ are built from classically precomputed constants.

Worked example 6.8: QPE with $t = 3$

Step 1. $\varphi = 3/8 = 0.011_2$: $y = 3$ (bits 011) with probability 1.

Step 2. $\varphi = 1/3$: $8\varphi = 2.67$; $\delta = 1/3 - 3/8 = -1/24$; $P(3) = \frac{\sin^2(\pi/3)}{64\sin^2(\pi/24)} = \frac{0.75}{64\times 0.01704} \approx 0.688$; $P(2) \approx 0.175$.

7 Continued fractions and classical post-processing

7.1 Why the measurement is not the period

y is random; it only satisfies $y/2^t \approx s/r$. If $\gcd(s, r) > 1$ the reduced fraction reveals only a divisor of r; $y = 0$ ($s = 0$) occurs for every r and carries no information. The quantum computer proposes candidates; the classical computer *verifies* them.

7.2 Continued fractions

Write $x = [a_0; a_1, a_2, \dots]$ with $a_0 = \lfloor x \rfloor$ and recursion on $1/(x - a_0)$. The truncations (convergents) are the best rational approximations. **Legendre's theorem:** if $|x - s/r| < 1/(2r^2)$ then s/r (in lowest terms) is a convergent of x. With $2^t \geq N^2$ and $r < N$, the outcome nearest a peak satisfies $|y/2^t - s/r| \leq 2^{-t-1} \leq 1/(2N^2) < 1/(2r^2)$. This is why $t \approx 2n$; the notebook's $t = 2n$ always satisfies $2^t \geq N^2$.

Method: Post-processing used by `Shor.ipynb` (`candidate_orders`)

1. If $y = 0$: no candidate.
2. $s/r' =$ `Fraction(y, 2**t).limit_denominator(N)` (closest fraction with denominator $\leq N$).
3. Take the smallest multiple kr' ($k = 1, 2, \dots$) with $a^{kr'} \equiv 1 \pmod N$ — this repairs $\gcd(s, r) > 1$.
4. Apply the reduction: r even, $x = a^{r/2} \not\equiv -1$, $p = \gcd(x - 1, N)$, $q = \gcd(x + 1, N)$; check $pq = N$.

Worked example 7.9: $y = 171$ for $N = 21$, $a = 2$, $t = 10$

Step 1. $171/1024 = 0.16699 = [0; 5, 1, 84, 2]$; convergents $0, \frac{1}{5}, \frac{1}{6}, \frac{85}{511}, \dots$

Step 2. The largest-denominator convergent with denominator ≤ 21 is $\frac{1}{6}$: $r' = 6$; $2^6 = 64 \equiv 1$, so $r = 6$.

Step 3. $2^3 = 8$: $\gcd(7, 21) = 7$, $\gcd(9, 21) = 3$.

Worked example 7.10: The multiples test: $y = 512$ for $N = 21$

$512/1024 = 1/2$ (here $s = 3$, $r = 6$, gcd $= 3$). Test $2k$: $2^2 = 4$, $2^4 = 16$, $2^6 = 64 \equiv 1$: $r = 6$. Without the test, a shot gives r directly only with probability about $\varphi(r)/r$ ($= \frac{1}{3}$ for $r = 6$ or 12).

Common trap

The numerator of the fraction is s, not r. A candidate must be verified ($a^r \equiv 1$); denominators larger than N are never orders.

8 Fully worked problems

Worked example 8.11: Factoring $N = 15$ with $a = 2$, step by step

Step 1. $\gcd(2, 15) = 1$: continue. $n = 4$, $t = 8$, 12 qubits.

Step 2. $f(x) = 2^x \bmod 15$: $1, 2, 4, 8, 1, 2, 4, 8, \ldots$ — period $r = 4$ (listed only because N is tiny; this table is what the quantum computer replaces).

Step 3. Constants $2^{2^j} \bmod 15$: $2, 4, 1, 1, \ldots$ — only U_2 and U_4 are applied.

Step 4. After the inverse QFT: $y \in \{0, 64, 128, 192\}$, each with probability $\frac{1}{4}$.

Step 5. $y = 64 \to \frac{1}{4}$, $y = 192 \to \frac{3}{4}$: $r = 4$. $y = 128 \to \frac{1}{2} \to$ multiples $\to 4$. $y = 0$: nothing.

Step 6. Verify $2^4 = 16 \equiv 1$; r even; $2^2 = 4 \not\equiv 14$.

Step 7. $\gcd(3, 15) = 3$, $\gcd(5, 15) = 5$; $3 \times 5 = 15$. Success probability per shot $\frac{3}{4}$.

Re-executed with the notebook's circuit (1000 shots, seed 42): success fraction 0.766 (exact 0.750). This is a classical simulation, not a quantum-hardware run.

Worked example 8.12: Factoring $N = 35$ with $a = 3$

Step 1. $\gcd(3, 35) = 1$; $n = 6$, $t = 12$, 18 qubits.

Step 2. $r = \mathrm{lcm}(\mathrm{ord}_5\, 3, \mathrm{ord}_7\, 3) = \mathrm{lcm}(4, 6) = 12$; peaks near $k \cdot 4096/12 = 341.33k$.

Step 3. An outcome $y = 1365$: $1365/4096 = 0.33325 \approx \frac{1}{3} \Rightarrow r' = 3$; $3^3 = 27 \neq 1$, $3^6 = 729 \equiv 29 \neq 1$, $3^9 \equiv 29 \cdot 27 = 783 \equiv 13$, $3^{12} \equiv 1$: $r = 12$.

Step 4. $3^6 \equiv 29 \not\equiv 34$: $\gcd(28, 35) = 7$, $\gcd(30, 35) = 5$.

Re-executed exact success probability per shot: 0.904 (5.64 s on the CPU for 1000 shots).

Worked example 8.13: Reading a distribution with a failing base

$N = 15$, $a = 14$: $r = 2$, outcomes 0 and 128. $y = 128$ gives $r = 2$ correctly, but $14 \equiv -1$: gcds 1 and 15. The success probability is 0 for *every* shot — the algorithm must change a, not take more shots.

9 Circuit architecture and resources

9.1 Register sizes and qubit counts

construction	logical qubits	RSA-2048	idea
`Shor.ipynb`	$3n$	6144	$t = 2n$ counting, n work, matrix multipliers
Beauregard (2003)	$2n + 3$	4099	semiclassical QFT, QFT-based adders
Gidney–Ekerå (2019)	$3n + 0.002n \log_2 n$	6190	windowed arithmetic, coset representation
idealised semiclassical	$n + 1$	2049	one recycled control, oracle multipliers

The **semiclassical QFT** (Griffiths–Niu) measures the counting qubits one at a time and applies the remaining phase corrections classically controlled, so one control qubit is reused $2n$ times. Real arithmetic needs **ancillas** (carries, temporary results) and uncomputation.

9.2 Gates, depth and connectivity

Modular exponentiation dominates: $t = 2n$ controlled multiplications, each $O(n^2)$ gates with schoolbook arithmetic, $O(n^3)$ in total (doubling n multiplies the cost by 8). The QFT costs only $O(t^2)$, or $O(t \log t)$ approximated. On hardware with limited connectivity, SWAP networks or lattice surgery add overhead.

Worked example 9.14: Counting gates of a gate-level multiplier for $N = 15$ (illustrative)

Multiplying by 2 modulo 15 is a cyclic rotation of the 4 work bits (three SWAPs); by $7 \equiv -8$: three rotations, then X on all bits (valid on $y = 1, \ldots, 14$). Controlled: 9 Fredkins (72 CNOTs) + 4 CNOTs = 76 CNOTs. With the controlled $\times 4$ (48 CNOTs) and the inverse QFT for $t = 3$ (one SWAP = 3 CNOTs, 3 controlled phases) the circuit has 127 CNOTs and 273 gates in total. Such shortcuts use the known answer and do not generalise.

10 The implementation in `Shor.ipynb`

10.1 Structure and code

The notebook validates N (even, prime, prime power are handled classically), picks a (`BASE_A`; invalid or lucky bases are reported and replaced by a random coprime base), computes n and t, builds the circuit with CUDA-Q's kernel builder, samples it, post-processes every outcome and retries with a new base up to `MAX_ATTEMPTS = 5` times. Hardware targets are refused because the multipliers are matrix gates.

```
def mulmod_matrix(c, modulus, n): # permutation |y> -> |c*y mod N>, identity for y >= N
    dim = 2 ** n
    matrix = np.zeros((dim, dim))
    for y in range(dim):
        matrix[(c * y) % modulus if y < modulus else y, y] = 1.0
    return matrix

kernel.x(work[n - 1]) # |1>: last work qubit = least significant bit
kernel.h(count)
for j in range(t):
    c = pow(a, 2 ** j, modulus) # classical repeated squaring
    if c == 1:
        continue # identity
```

```
    name = f"mulmod_{c}_mod_{modulus}"
    cudaq.register_operation(name, mulmod_matrix(c, modulus, n)) # once per name
    getattr(kernel, name)(count[t - 1 - j], *[work[i] for i in range(n)])
# inverse QFT: swaps, then cr1(-pi/2**(k-j)) and h; finally kernel.mz(count)
```

`count[0]` is the most significant bit of y, so counting qubit $t-1-j$ controls $U_{a^{2^j}}$.

10.2 Results

Previously recorded run (`Shor.ipynb`, CUDA-Q 0.16.0, `qpp-cpu`, fp64, no GPU): $N = 15$, $a = 7$, 12 qubits, 51 gates plus 8 measurements (`h`: 16, `cr1`: 28, `swap`: 4, `x`: 1, controlled custom multipliers: 2), depth 20; 1000 shots gave $y = 192, 128, 0, 64$ with counts 272, 268, 234, 226; 0.766 of the shots yielded $15 = 3 \times 5$; whole execution cell 0.073 s.

Re-executed cases (same builder, `make_figures.py`): the recorded counts were reproduced (identical to the recorded outputs).

N	a	r	qubits	U gates	distinct y (1000 shots)	P_{succ} exact	from 1000 shots	factors
15	7	4	$8+4=12$	2	4	0.750	0.766	3×5
15	2	4	$8+4=12$	2	4	0.750	0.766	3×5
15	14	2	$8+4=12$	1	2	0.000	0.000	none
21	2	6	$10+5=15$	10	54	0.817	0.822	3×7
35	3	12	$12+6=18$	12	76	0.904	0.897	5×7

Reverse estimation (recorded): 100 logical qubits allow 33-bit N with $3n$ and 48-bit N with $2n+3$; an 80 GB memory budget allows a 32-qubit state vector, i.e. $N \leq 1023$.

Common trap

Simulation verifies the circuit; it does not demonstrate a quantum speed-up — the simulator itself does work exponential in the number of qubits.

11 Noise, approximations and limitations

11.1 Noisy and approximate circuits (illustrative experiments)

The gate-level $N = 15$ circuit ($t = 3$) was run on `density-matrix-cpu` with a one-qubit depolarizing channel p after every H, R_z, X and a two-qubit depolarizing channel p after every CNOT and controlled phase. The success probability per shot falls from 0.750 to 0.703 at $p = 0.01$ and 0.665 at $p = 0.03$, towards the **random baseline** 0.375 (uniformly random y: $y = 2, 4, 6$ succeed). Note that $0.99^{127} \approx 0.28$ is much smaller than 0.703: many errors do not change the answer.

Approximate inverse QFT for $N = 21$, $t = 10$ (keep controlled phases between qubits at distance $\leq m$):

max. $k-j$ kept	controlled phases	P_{succ}
1	9	0.4397
2	17	0.6892
3	24	0.7865
4	30	0.8105
9	45	0.8165

11.2 Limitations

- **Qubits**: thousands of logical qubits for RSA-2048; each logical qubit needs hundreds to thousands of physical qubits.
- **Gates**: billions of Toffoli gates; non-Clifford gates require magic states, which dominate the cost.
- **Errors**: physical error rates $\sim 10^{-3}$ must be suppressed by error correction (surface code: $\approx 2d^2$ physical qubits per logical qubit, logical error $\sim (p/p_{\text{th}})^{(d+1)/2}$).
- **Depth, connectivity, real-time decoding** and mid-circuit measurement for the semiclassical QFT.
- **Published estimates** (not experiments): Gidney–Ekerå 2019, ≈ 20 million noisy qubits and ≈ 8 hours; Gidney 2025, fewer than one million noisy qubits and less than a week (physical error rate 10^{-3}, $1\,\mu$s cycle).

Common trap

Demonstrations that "factor 15 or 21" use a handful of qubits and often circuits simplified with knowledge of the answer (Smolin, Smith and Vargo, 2013). They say nothing about the feasibility of factoring RSA-2048.

12 Cryptography and outlook

Shor's period finding solves factoring and discrete logarithms in any abelian group: RSA, Diffie–Hellman, DSA, ECDH and ECDSA are broken by a large fault-tolerant quantum computer (elliptic curves need fewer qubits than RSA of equal classical strength). Symmetric ciphers and hash functions only face Grover's square-root speed-up; AES-256 remains adequate. **Post-quantum cryptography** replaces the public-key schemes with classical algorithms believed to resist quantum attacks: NIST FIPS 203 (ML-KEM), FIPS 204 (ML-DSA) and FIPS 205 (SLH-DSA), published in August 2024. Organisations migrate now because of **harvest-now-decrypt-later**: data recorded today could be decrypted later, and migrations take years (Mosca: act if data lifetime plus migration time exceeds the time to a cryptographically relevant quantum computer).

Research directions: cheaper arithmetic (windowing, approximate residue arithmetic), better codes and magic-state production, Ekerå–Håstad and Regev-type variants, elliptic-curve discrete logarithms, and honest benchmarking.

13 Problem-solving strategies and question map

13.1 Strategy by question format

- **Single-correct numerical:** compute fully, then sanity-check: orders divide $\lambda(N)$; probabilities sum to 1; denominators $\leq N$; qubits $= 3n$ in the notebook. Typical distractors: "a multiple of the order", "$\varphi(N)$ instead of the order", "amplitude instead of probability", "numerator instead of denominator", "forgot the work register".
- **Matching lists:** solve the most definite row first, then eliminate options.
- **Assertion–Reason:** judge A and R *independently*; only if both are true ask whether R *explains* A.
- **Statement I/II:** one counterexample ($y = 0$, $a = N-1$, $N = p^k$) kills a "for every" statement.

- **Paragraph:** the first question computes r; the second reuses it for gcds or continued fractions.

13.2 Fast checks that catch most mistakes

1. Is $\gcd(a, N) = 1$? Is N a prime power?
2. Verify every candidate order by $a^r \equiv 1$ and check that no smaller divisor works.
3. $a^{r/2} \equiv -1$? Then the attempt fails.
4. Multiply the factors back together.
5. Recorded vs. re-executed vs. illustrative vs. estimated numbers — which one does the question use?

13.3 Map: assessment questions → sections of this chapter

Questions	Where to learn it
Q.1–Q.10, Q.12, Q.15, Q.18	Section 1 (gcd, inverses, totient, order, repeated squaring, CRT, square roots of 1)
Q.13, Q.16, Q.72	Section 2 (classical factoring, complexity, RSA)
Q.11, Q.14, Q.17, Q.19, Q.20, Q.41–Q.45, Q.57, Q.59	Section 3 (reduction, success and failure cases, prime powers)
Q.21–Q.28, Q.30, Q.31, Q.36, Q.37	Section 4 (registers, unitarity, eigenvectors, kickback, gates)
Q.29, Q.32, Q.35, Q.38, Q.47, Q.51, Q.53, Q.66	Section 5 (period finding, interference, QFT)
Q.39, Q.40	Section 6 (phase estimation)
Q.48–Q.50, Q.52, Q.56, Q.58, Q.60	Section 7 (continued fractions, post-processing)
Q.42, Q.47, Q.48, Q.59	Section 8 (worked problems)
Q.33, Q.34, Q.46, Q.61, Q.63, Q.64, Q.69, Q.71, Q.76, Q.78	Section 9 (qubits, memory, gates, constructions)
Q.54, Q.55, Q.67, Q.68	Section 10 (the notebook's code and recorded results)
Q.62, Q.65, Q.70, Q.75, Q.79, Q.80	Section 11 (estimates, approximate QFT, noise, fault tolerance)
Q.73, Q.74, Q.77	Section 12 (cryptography, post-quantum standards)

14 Practice problems

Try each before looking at the answers on the next page. They use the same ideas as the assessment but different numbers.

P1. $\gcd(252, 198)$ by the Euclidean algorithm.

P2. The inverse of 5 modulo 17.

P3. $\varphi(77)$.

P4. The order of 3 modulo 7.

P5. $5^{21} \bmod 11$ (use the order of 5).

P6. All square roots of 1 modulo 21.

P7. $N = 35$, $a = 2$: the order, and the factors from the reduction.

P8. Qubits and state-vector memory of the notebook's circuit for $N = 55$.

P9. $N = 35$, $a = 2$, $t = 12$: the outcome $y = 3072$ is measured. What does the post-processing return?

P10. `Fraction(205, 512).limit_denominator(21)`.

P11. For $r = 10$ dividing 2^t, the fraction of shots whose reduced fraction has denominator r (no multiples test).

P12. QPE with $t = 2$ and $\varphi = 1/3$: the most likely outcome and its probability.

P13. Beauregard's qubit count for RSA-3072.

P14. The number of controlled phase gates in a full QFT on 12 qubits.

P15. Toy RSA with $p = 3$, $q = 11$, $e = 3$: d, and the ciphertext of $m = 4$.

P16. A report says "a two-qubit device factored 35 with Shor's algorithm". What do you conclude?

Answers to the practice problems

P1. $252 = 1 \cdot 198 + 54$, $198 = 3 \cdot 54 + 36$, $54 = 1 \cdot 36 + 18$, $36 = 2 \cdot 18$: gcd $= 18$.

P2. $5 \cdot 7 = 35 = 2 \cdot 17 + 1$: $5^{-1} \equiv 7$.

P3. $(7-1)(11-1) = 60$.

P4. $3, 2, 6, 4, 5, 1$: order 6 (3 is a primitive root modulo 7).

P5. $5^2 \equiv 3$, $5^4 \equiv 9$, $5^5 \equiv 45 \equiv 1$: order 5; $21 = 4 \cdot 5 + 1$, so $5^{21} \equiv 5$.

P6. $1, 8, 13, 20$ ($8^2 = 64 = 3 \cdot 21 + 1$).

P7. $\operatorname{ord}_5 2 = 4$, $\operatorname{ord}_7 2 = 3$: $r = 12$; $2^6 = 64 \equiv 29 \not\equiv 34$; $\gcd(28, 35) = 7$, $\gcd(30, 35) = 5$.

P8. $55 = 110111_2$: $n = 6$, 18 qubits; $16 \cdot 2^{18} = 4\,194\,304$ bytes ≈ 4.2 MB.

P9. $3072/4096 = 3/4$: $r' = 4$; $2^4 = 16$, $2^8 \equiv 11$, $2^{12} \equiv 1$: the multiples test gives $r = 12$, then the factors 7 and 5 as in P7.

P10. $205/512 = 0.40039$; the closest fraction with denominator ≤ 21 is $2/5$.

P11. $\varphi(10)/10 = 4/10 = 0.4$ ($s = 1, 3, 7, 9$).

P12. $4\varphi = 1.33$: $y = 1$ with $P \approx 0.70$ (then $y = 2$: 0.19).

P13. $2 \cdot 3072 + 3 = 6147$ logical qubits.

P14. $\binom{12}{2} = 66$.

P15. $\varphi(33) = 20$, $3 \cdot 7 = 21 \equiv 1$: $d = 7$; $c = 4^3 = 64 \equiv 31 \pmod{33}$ (check: $31^7 \equiv 4$).

P16. Two qubits cannot hold the registers for $N = 35$ (the notebook would need 18); the circuit must have been compiled using the known period or factors. It is a demonstration of a few gates, not evidence that Shor's algorithm scales.

Summary of the chapter in ten lines

1. Factoring reduces classically to finding the order r of a random coprime a modulo N.
2. If r is even and $a^{r/2} \not\equiv -1$, then $\gcd(a^{r/2} \pm 1, N)$ are the factors; otherwise repeat (success $\geq \frac{1}{2}$ per base for RSA moduli).
3. Even N, primes, prime powers and lucky bases are handled classically.
4. U_a is a permutation (unitary) because $\gcd(a, N) = 1$; its eigenphases are s/r.
5. Circuit: $H^{\otimes t}$, controlled $U_{a^{2^j}}$ (repeated squaring), inverse QFT, measure y.
6. Outcomes peak at $y \approx k2^t/r$; exact peaks when $r \mid 2^t$; $y = 0$ is uninformative.
7. Continued fractions turn $y/2^t$ into s/r; verify $a^r \equiv 1$, test multiples, then take gcds.
8. The notebook uses $3n$ qubits and matrix multipliers: fine for simulation ($N \leq 1023$ in 80 GB), impossible on hardware.
9. Real implementations need $\approx 2n$–$3n$ logical and 10^6–10^7 physical qubits for RSA-2048 (estimates).
10. RSA and ECC must migrate to post-quantum cryptography (FIPS 203/204/205); no RSA-size number has been factored by a quantum computer.